DOCUMENTO BASE DEL PROYECTO

OPENTALK WISP

CRM Omnicanal Híbrido con IA para WhatsApp

💼 INFORMACIÓN GENERAL DEL PROYECTO

Nombre del Proyecto

WhatsApp CRM AI - Sistema de Gestión de Relaciones con Clientes Omnicanal con Inteligencia Artificial

Versión

1.0.0 - Fase de Desarrollo Inicial

Fecha de Creación

9 de diciembre de 2025

Tipo de Proyecto

SaaS (Software as a Service) - CRM Omnicanal con Automatización por IA

🎯 VISIÓN Y OBJETIVOS

Visión General

Crear un sistema CRM empresarial que centralice las comunicaciones de WhatsApp (y futuros canales) permitiendo a las empresas gestionar ventas, atención al cliente y marketing de manera inteligente mediante automatización con IA.

Objetivos Principales

1. **Conectividad Híbrida**

- Soportar WhatsApp Cloud API (Meta) - Conexión oficial
- Soportar WhatsApp Web (QR) - Conexión no oficial mediante Baileys/WWebJS
- Arquitectura que permita cambiar entre proveedores sin afectar la lógica de negocio

2. **Inteligencia Artificial Integrada**

- Respuestas automáticas contextuales usando OpenAI/LangChain
- Constructor de flujos conversacionales mediante lenguaje natural
- Lead Scoring automático
- Análisis de sentimiento en tiempo real
- Generación dinámica de chatbots sin código

3. **Gestión CRM Completa**

- Base de datos de contactos con historial completo
- Pipeline de ventas visual (Kanban)
- Etiquetado y segmentación de leads
- Métricas y analytics en tiempo real

4. **Campañas y Broadcasting**

- Envío masivo de mensajes multimedia
- Programación de campañas
- Segmentación avanzada de audiencias
- Tracking de conversiones

5. **Escalabilidad Omnicanal**

- Arquitectura preparada para Facebook Messenger
- Arquitectura preparada para Instagram DM
- Sistema de adaptadores para futuros canales

6. **Integraciones Externas**

- API REST documentada
- Webhooks para eventos
- Compatibilidad con Zapier/Make/n8n

🏗️ ARQUITECTURA DEL SISTEMA

Patrón Arquitectónico Principal

Clean Architecture + Adapter Pattern

Capas de la Arquitectura

Copiar

```
┌─────────────────────────────────────────┐
│         CAPA DE PRESENTACIÓN            │
│    (Frontend - Next.js + React)         │
└─────────────────────────────────────────┘
                    ↓
┌─────────────────────────────────────────┐
│         CAPA DE APLICACIÓN              │
│    (Controladores, DTOs, Validación)    │
└─────────────────────────────────────────┘
                    ↓
┌─────────────────────────────────────────┐
│         CAPA DE DOMINIO                 │
│    (Lógica de Negocio, Entidades)       │
└─────────────────────────────────────────┘
                    ↓
┌─────────────────────────────────────────┐
│         CAPA DE INFRAESTRUCTURA         │
│    (Adaptadores, DB, APIs Externas)     │
└─────────────────────────────────────────┘
```

Patrón Adapter para Proveedores de WhatsApp

Copiar

```typescript
// Interfaz unificada
interface IWhatsAppProvider {
  connect(): Promise<void>;
  sendMessage(to: string, message: Message): Promise<void>;
  onMessage(callback: (msg: UnifiedMessage) => void): void;
  disconnect(): Promise<void>;
}

// Implementaciones
class MetaCloudProvider implements IWhatsAppProvider { }
class BaileysQRProvider implements IWhatsAppProvider { }
class WWebJSProvider implements IWhatsAppProvider { }
```

💻 STACK TECNOLÓGICO

Backend

Tecnología	Versión	Propósito
Node.js	20.x LTS	Runtime principal
NestJS	10.x	Framework backend (arquitectura modular)
TypeScript	5.x	Lenguaje tipado
PostgreSQL	15.x	Base de datos relacional (CRM, usuarios)
Redis	7.x	Cache, sesiones, colas de mensajes
Prisma	5.x	ORM para PostgreSQL
Bull	4.x	Sistema de colas (mensajes masivos)

Frontend

Tecnología	Versión	Propósito
Next.js	14.x	Framework React con SSR
React	18.x	Librería UI
TypeScript	5.x	Tipado estático
Tailwind CSS	3.x	Framework CSS utility-first
Zustand	4.x	Gestión de estado global
React Query	5.x	Manejo de datos asíncronos
Shadcn/ui	Latest	Componentes UI accesibles

Inteligencia Artificial

Tecnología	Propósito
OpenAI API	Generación de respuestas, análisis
LangChain	Orquestación de LLMs, memoria conversacional
Pinecone/Qdrant	Vector database para RAG (opcional)

WhatsApp Integration

Librería	Propósito
@whiskeysockets/baileys	Conexión WhatsApp Web (QR)
whatsapp-web.js	Alternativa para conexión QR
@meta/whatsapp-business	API oficial de Meta

DevOps & Infraestructura

Tecnología	Propósito
Docker	Contenedorización
Docker Compose	Orquestación local
Nginx	Reverse proxy
PM2	Process manager (producción)
GitHub Actions	CI/CD

📁 ESTRUCTURA DE DIRECTORIOS

Copiar

```
whatsapp-crm-ai/
├── apps/
│   ├── backend/                    # Aplicación NestJS
│   │   ├── src/
│   │   │   ├── main.ts
│   │   │   ├── app.module.ts
│   │   │   ├── core/               # Capa de dominio
│   │   │   │   ├── entities/       # Entidades de negocio
│   │   │   │   ├── interfaces/     # Contratos
│   │   │   │   └── use-cases/      # Casos de uso
│   │   │   ├── modules/
│   │   │   │   ├── auth/           # Autenticación
│   │   │   │   ├── contacts/       # Gestión de contactos
│   │   │   │   ├── conversations/  # Chats
│   │   │   │   ├── campaigns/      # Broadcasting
│   │   │   │   ├── pipeline/       # Embudo de ventas
│   │   │   │   ├── flows/          # Constructor de flujos IA
│   │   │   │   ├── ai/             # Servicios de IA
│   │   │   │   └── whatsapp/       # Módulo WhatsApp
│   │   │   │       ├── providers/  # Adaptadores
│   │   │   │       │   ├── meta-cloud.provider.ts
│   │   │   │       │   ├── baileys.provider.ts
│   │   │   │       │   └── provider.interface.ts
│   │   │   │       ├── services/
│   │   │   │       └── controllers/
```

```
│   │   │   ├── infrastructure/
│   │   │   │   ├── database/       # Prisma, migraciones
│   │   │   │   ├── cache/          # Redis
│   │   │   │   ├── queues/         # Bull queues
│   │   │   │   └── external-apis/  # Integraciones
│   │   │   └── shared/
│   │   │       ├── decorators/
│   │   │       ├── filters/
│   │   │       ├── guards/
│   │   │       ├── interceptors/
│   │   │       └── utils/
│   │   ├── prisma/
│   │   │   └── schema.prisma
│   │   ├── test/
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   └── frontend/                   # Aplicación Next.js
│       ├── src/
│       │   ├── app/                # App Router (Next.js 14)
│       │   │   ├── (auth)/         # Rutas de autenticación
│       │   │   ├── (dashboard)/    # Rutas del dashboard
│       │   │   │   ├── contacts/
│       │   │   │   ├── conversations/
│       │   │   │   ├── pipeline/
│       │   │   │   ├── campaigns/
│       │   │   │   ├── flows/
│       │   │   │   └── analytics/
│       │   │   ├── layout.tsx
│       │   │   └── page.tsx
│       │   ├── components/
│       │   │   ├── ui/             # Componentes base (Shadcn)
│       │   │   ├── layout/         # Sidebar, Header
│       │   │   ├── contacts/
│       │   │   ├── chat/
│       │   │   ├── pipeline/
│       │   │   └── flows/
│       │   ├── lib/
│       │   │   ├── api/            # Cliente API
│       │   │   ├── hooks/          # Custom hooks
│       │   │   ├── stores/         # Zustand stores
│       │   │   └── utils/
│       │   ├── types/
│       │   └── styles/
│       ├── public/
│       ├── package.json
│       └── next.config.js
│
├── packages/                       # Código compartido
│   ├── shared-types/               # Types compartidos
│   ├── shared-utils/               # Utilidades compartidas
│   └── config/                     # Configuraciones
│
├── docker/
│   ├── Dockerfile.backend
│   ├── Dockerfile.frontend
│   └── docker-compose.yml
│
├── docs/                           # Documentación
```

```
│   ├── api/                        # Documentación API
│   ├── architecture/               # Diagramas
│   └── deployment/                 # Guías de despliegue
│
├── scripts/                        # Scripts de utilidad
│   ├── setup.sh
│   └── migrate.sh
│
├── .github/
│   └── workflows/
│       └── ci-cd.yml
│
├── .gitignore
├── .env.example
├── package.json                    # Root package (monorepo)
├── turbo.json                      # Configuración Turborepo
├── pnpm-workspace.yaml             # Workspaces
└── README.md
```

🎁 MÓDULOS FUNCIONALES

1. Módulo de Autenticación (Auth)

Responsabilidades:

- Registro y login de usuarios
- JWT tokens
- Roles y permisos (Admin, Agente, Supervisor)
- Multi-tenancy (cada empresa es un tenant)

Tecnologías:

- Passport.js
- JWT
- Bcrypt

---

2. Módulo de Conexión WhatsApp (WhatsApp Provider)

Responsabilidades:

- Implementar patrón Adapter
- Gestionar conexiones múltiples (QR y Meta)

- Unificar eventos de mensajes entrantes
- Manejo de sesiones

Componentes Clave:

Copiar

```
// provider.interface.ts
export interface IWhatsAppProvider {
  connect(config: ConnectionConfig): Promise<ConnectionStatus>;
  disconnect(): Promise<void>;
  sendTextMessage(to: string, text: string): Promise<MessageResult>;
  sendMediaMessage(to: string, media: MediaMessage): Promise<MessageResult>;
  onMessage(handler: (msg: UnifiedMessage) => void): void;
  onStatusChange(handler: (status: ConnectionStatus) => void): void;
  getQRCode?(): Promise<string>; // Solo para providers QR
}

// Mensaje unificado
export interface UnifiedMessage {
  id: string;
  from: string;
  to: string;
  timestamp: Date;
  type: 'text' | 'image' | 'video' | 'audio' | 'document';
  content: string | MediaContent;
  context?: MessageContext; // Para respuestas
  provider: 'meta' | 'baileys' | 'wwebjs';
}
```

---

3. Módulo de Contactos (Contacts)

Responsabilidades:

- CRUD de contactos
- Historial de interacciones
- Etiquetado y segmentación
- Importación/Exportación (CSV, Excel)
- Deduplicación automática

Entidades:

- Contact
- ContactTag
- ContactNote
- ContactCustomField

---

4. Módulo de Conversaciones (Conversations)

Responsabilidades:

- Gestión de chats en tiempo real
- Asignación de conversaciones a agentes
- Estados de conversación (Abierta, En espera, Cerrada)
- Búsqueda en historial
- Notas internas

Tecnologías:

- WebSockets (Socket.io)
- Redis Pub/Sub

---

5. Módulo de Pipeline de Ventas (Pipeline)

Responsabilidades:

- Tablero Kanban visual
- Etapas personalizables
- Drag & drop de leads
- Automatizaciones por etapa
- Reportes de conversión

Entidades:

- Pipeline
- PipelineStage
- Deal
- DealActivity

---

6. Módulo de Campañas (Campaigns)

Responsabilidades:

- Creación de campañas de broadcasting
- Segmentación de audiencias
- Programación de envíos
- Plantillas de mensajes
- Tracking de métricas (enviados, entregados, leídos, respuestas)

Componentes:

- Campaign Manager
- Message Queue (Bull)
- Rate Limiter (para no ser bloqueado)

---

7. Módulo de Inteligencia Artificial (AI)

Responsabilidades:

- Generación de respuestas automáticas
- Análisis de sentimiento
- Lead Scoring
- Resumen de conversaciones
- Sugerencias de respuesta para agentes

Servicios:

Copiar

```
// ai.service.ts
export class AIService {
  async generateResponse(
    context: ConversationContext,
    userMessage: string
  ): Promise<string>;

  async analyzeSentiment(message: string): Promise<SentimentResult>;

  async scoreLead(contact: Contact, history: Message[]): Promise<number>;

  async summarizeConversation(messages: Message[]): Promise<string>;

  async suggestNextAction(deal: Deal): Promise<ActionSuggestion>;
}
```

Integración con LangChain:

Copiar

```typescript
// langchain-flow.service.ts
export class LangChainFlowService {
  async createFlowFromPrompt(prompt: string): Promise<Flow>;

  async executeFlow(
    flowId: string,
    input: FlowInput
  ): Promise<FlowOutput>;

  async updateFlowMemory(
    sessionId: string,
    interaction: Interaction
  ): Promise<void>;
}
```

---

8. Módulo de Constructor de Flujos (Flows)

Responsabilidades:

- Crear flujos conversacionales mediante IA
- Editor visual de flujos (opcional)
- Nodos de decisión, acciones, condiciones
- Testing de flujos
- Versionado de flujos

Tipos de Nodos:

- Mensaje (enviar texto/media)
- Pregunta (esperar respuesta)
- Condición (if/else)
- Acción (actualizar CRM, crear deal, etc.)
- IA (delegar a LLM)
- Webhook (llamar API externa)

---

9. Módulo de API REST (API)

Responsabilidades:

- Endpoints documentados (Swagger)
- Webhooks para eventos

- Rate limiting
- API Keys para clientes
- Logs de peticiones

Endpoints Principales:

Copiar

```
POST   /api/v1/messages/send
GET    /api/v1/contacts
POST   /api/v1/contacts
GET    /api/v1/conversations/:id
POST   /api/v1/campaigns
GET    /api/v1/analytics/dashboard
POST   /api/v1/webhooks/register
```

🔐 SEGURIDAD

Medidas Implementadas

1. **Autenticación y Autorización**
- JWT con refresh tokens
- RBAC (Role-Based Access Control)
- Multi-factor authentication (opcional)

2. **Protección de Datos**
- Encriptación de credenciales (bcrypt)
- Encriptación de datos sensibles en DB
- HTTPS obligatorio
- Variables de entorno para secrets

3. **Rate Limiting**
- Límites por IP
- Límites por usuario
- Límites por API key

4. **Validación de Datos**
- DTOs con class-validator
- Sanitización de inputs

- Protección contra SQL injection (Prisma ORM)
- Protección contra XSS

5. **Monitoreo**

- Logs estructurados (Winston)
- Alertas de seguridad
- Auditoría de acciones críticas

🚀 ROADMAP DE DESARROLLO

Fase 1: MVP (Meses 1-2)

- [ ] Configuración del monorepo
- [ ] Autenticación básica
- [ ] Conexión WhatsApp (solo QR - Baileys)
- [ ] CRUD de contactos
- [ ] Chat básico en tiempo real
- [ ] Envío/recepción de mensajes
- [ ] Dashboard simple

Fase 2: CRM Core (Meses 3-4)

- [ ] Pipeline de ventas (Kanban)
- [ ] Etiquetado de contactos
- [ ] Asignación de conversaciones
- [ ] Notas y actividades
- [ ] Búsqueda avanzada
- [ ] Reportes básicos

Fase 3: IA Básica (Mes 5)

- ☐ Integración con OpenAI
- ☐ Respuestas automáticas simples
- ☐ Análisis de sentimiento
- ☐ Sugerencias de respuesta

Fase 4: Campañas (Mes 6)

- ☐ Broadcasting masivo
- ☐ Plantillas de mensajes
- ☐ Programación de envíos
- ☐ Sistema de colas con Bull
- ☐ Métricas de campaña

Fase 5: IA Avanzada (Meses 7-8)

- ☐ Integración LangChain
- ☐ Constructor de flujos con IA
- ☐ Lead Scoring automático
- ☐ RAG para conocimiento empresarial
- ☐ Chatbots personalizados

Fase 6: Escalabilidad (Mes 9)

- ☐ Conexión Meta Cloud API
- ☐ Multi-instancia WhatsApp
- ☐ Optimización de performance
- ☐ Caché avanzado
- ☐ Microservicios (opcional)

Fase 7: Omnicanalidad (Meses 10-12)

- ☐ Facebook Messenger

- ☐ Instagram DM
- ☐ Bandeja unificada
- ☐ Routing inteligente

Fase 8: Integraciones (Mes 12+)

- ☐ API REST completa
- ☐ Webhooks
- ☐ Zapier/Make
- ☐ Integraciones nativas (Shopify, WooCommerce, etc.)

📊 MÉTRICAS Y KPIs

Métricas del Sistema

- Tiempo de respuesta promedio
- Uptime del servicio
- Mensajes procesados por segundo
- Tasa de error de envío

Métricas de Negocio

- Tasa de conversión del pipeline
- Tiempo promedio de cierre de venta
- Satisfacción del cliente (CSAT)
- Tasa de respuesta de campañas
- ROI de campañas

Métricas de IA

- Precisión de respuestas automáticas
- Tasa de intervención humana
- Precisión del Lead Scoring
- Sentimiento promedio de conversaciones

🔧 CONFIGURACIÓN Y VARIABLES DE ENTORNO

Backend (.env)

Copiar

```
# Application
NODE_ENV=development
PORT=3000
APP_URL=http://localhost:3000

# Database
DATABASE_URL=postgresql://user:password@localhost:5432/whatsapp_crm
REDIS_URL=redis://localhost:6379

# JWT
JWT_SECRET=your-super-secret-key
JWT_EXPIRATION=7d
JWT_REFRESH_SECRET=your-refresh-secret
JWT_REFRESH_EXPIRATION=30d

# WhatsApp Meta
META_APP_ID=your-app-id
META_APP_SECRET=your-app-secret
META_PHONE_NUMBER_ID=your-phone-id
META_ACCESS_TOKEN=your-access-token
META_WEBHOOK_VERIFY_TOKEN=your-verify-token

# OpenAI
OPENAI_API_KEY=sk-your-api-key
OPENAI_MODEL=gpt-4-turbo-preview

# LangChain (opcional)
LANGCHAIN_API_KEY=your-langchain-key
PINECONE_API_KEY=your-pinecone-key

# Email (para notificaciones)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-password

# Storage (para archivos multimedia)
AWS_ACCESS_KEY_ID=your-access-key
AWS_SECRET_ACCESS_KEY=your-secret-key
AWS_S3_BUCKET=your-bucket-name
AWS_REGION=us-east-1
```

Frontend (.env.local)

Copiar

```
NEXT_PUBLIC_API_URL=http://localhost:3000/api
NEXT_PUBLIC_WS_URL=ws://localhost:3000
NEXT_PUBLIC_APP_NAME=WhatsApp CRM AI
```

📚 DOCUMENTACIÓN ADICIONAL REQUERIDA

Documentos Pendientes

1. **Modelo de Base de Datos** (PRÓXIMO)
- Esquema completo de Prisma
- Relaciones entre entidades
- Índices y optimizaciones
- Estrategia de migraciones

2. **Guía de API**
- Especificación OpenAPI/Swagger
- Ejemplos de peticiones
- Códigos de error
- Rate limits

3. **Manual de Despliegue**
- Configuración de servidores
- Docker compose para producción
- CI/CD con GitHub Actions
- Monitoreo y logs

4. **Guía de Contribución**
- Estándares de código
- Flujo de Git
- Testing requirements
- Code review process

☐ ESTRATEGIA DE TESTING

Tipos de Tests

1. **Unit Tests**
- Servicios individuales
- Utilidades
- Validadores
- Coverage objetivo: 80%

2. **Integration Tests**
- Endpoints de API
- Flujos de autenticación
- Integración con WhatsApp
- Integración con IA

3. **E2E Tests**
- Flujos críticos de usuario
- Playwright para frontend
- Supertest para backend

Herramientas

- Jest (unit + integration)
- Playwright (E2E frontend)
- Supertest (E2E backend)
- Testing Library (componentes React)

📞 SOPORTE Y MANTENIMIENTO

Estrategia de Versionado

Semantic Versioning (SemVer): MAJOR.MINOR.PATCH

Ciclo de Actualizaciones

- **Patches:** Cada 2 semanas (bugs críticos)
- **Minor:** Mensual (nuevas features)

- **Major:** Trimestral (breaking changes)

Canales de Soporte

- Documentación en línea
- Sistema de tickets
- Chat en vivo (usando el propio CRM ☺)
- Base de conocimiento

✅ CHECKLIST DE INICIO

Antes de comenzar el desarrollo, asegúrate de tener:

- ☐ Node.js 20+ instalado
- ☐ Docker y Docker Compose instalados
- ☐ PostgreSQL configurado (o vía Docker)
- ☐ Redis configurado (o vía Docker)
- ☐ Cuenta de OpenAI con API key
- ☐ Cuenta de Meta Developers (para fase posterior)
- ☐ Editor de código (VS Code recomendado)
- ☐ Git configurado
- ☐ Repositorio creado en GitHub/GitLab

🎓 RECURSOS DE APRENDIZAJE

Documentación Oficial

- NestJS Docs
- Next.js Docs
- Prisma Docs
- WhatsApp Business API

- Baileys GitHub
- LangChain Docs

Tutoriales Recomendados

- Clean Architecture en Node.js
- Adapter Pattern en TypeScript
- WebSockets con Socket.io
- Bull Queue para mensajería
- LangChain para chatbots

📝 NOTAS FINALES

Consideraciones Importantes

1. **Límites de WhatsApp**
- La API oficial tiene límites de mensajes por día
- La conexión QR puede ser bloqueada si se detecta uso comercial
- Implementar rate limiting es CRÍTICO

2. **Escalabilidad**
- Redis es fundamental para manejar múltiples conexiones
- Considerar microservicios si se superan 10,000 usuarios activos
- Implementar CDN para archivos multimedia

3. **Costos Estimados**
- OpenAI API: Variable según uso
- WhatsApp Cloud API: $0.005-0.009 por mensaje
- Servidor: $50-200/mes (dependiendo de escala)
- Base de datos: $20-100/mes

4. **Aspectos Legales**
- Cumplir con GDPR/LGPD para datos de usuarios
- Términos de servicio de WhatsApp
- Política de privacidad clara

- Consentimiento para mensajes automatizados

🚦 PRÓXIMOS PASOS

Inmediatos (Esta Semana)

1. **Configurar el Monorepo**
   Copiar

```
mkdir whatsapp-crm-ai
cd whatsapp-crm-ai
pnpm init
# Configurar workspaces
```

2. **Inicializar Backend**
   Copiar

```
nest new apps/backend
cd apps/backend
pnpm add @prisma/client prisma
pnpm add @nestjs/jwt @nestjs/passport passport passport-jwt
pnpm add @whiskeysockets/baileys
```

3. **Inicializar Frontend**
   Copiar

```
npx create-next-app@latest apps/frontend --typescript --tailwind --app
cd apps/frontend
pnpm add zustand @tanstack/react-query
```

4. **Configurar Docker**

- Crear docker-compose.yml con PostgreSQL y Redis
- Levantar servicios locales

5. **Solicitar el Modelo de Base de Datos**

- Este será tu próximo documento
- Incluirá el schema.prisma completo

📄 CONTROL DE VERSIONES DEL DOCUMENTO

Versión	Fecha	Cambios	Autor
1.0.0	2025-12-09	Documento inicial completo	Sistema IA

📧 CONTACTO DEL PROYECTO

Desarrollador Principal: [Tu Nombre]
Email: [tu-email]
Repositorio: [URL del repo]
Documentación: [URL de docs]

🎉 ¡DOCUMENTO BASE COMPLETADO!

Este documento servirá como la **biblia del proyecto**. Todos los desarrolladores, diseñadores y stakeholders deben leerlo antes de contribuir.

¿Qué sigue?

Ahora necesitas el **Modelo de Base de Datos**. Cuando estés listo, dime:

> "Genera el documento del modelo de base de datos con el schema de Prisma completo"

Y te crearé el archivo con:

- ✅ Todas las entidades y relaciones
- ✅ Schema de Prisma listo para usar
- ✅ Migraciones iniciales
- ✅ Seeders de ejemplo
- ✅ Diagramas ER

¿Estás listo para continuar con la base de datos? 🚀